



# 贝锐

## 向日葵 iOS 主控 SDK v3.0 接入说明

### 目录

|                          |       |
|--------------------------|-------|
| 向日葵 iOS 主控 SDK v3.0 接入说明 | ..... |
| 一、概述                     | ..... |
| 二、SDK 文件说明               | ..... |
| 三、集成系统要求                 | ..... |
| 四、集成步骤                   | ..... |
| 4.1 添加 framework 动态库     | ..... |
| 4.2 添加权限                 | ..... |
| 五、使用说明                   | ..... |
| 5.2 远程桌面                 | ..... |
| 5.2.1 配置桌面UI             | ..... |
| 5.2.2 连接远程桌面             | ..... |
| 5.2.3 远程桌面控制API          | ..... |
| 远程桌面状态                   | ..... |
| 基础控制                     | ..... |
| 桌面模式                     | ..... |
| 操作模式                     | ..... |
| 虚拟鼠标                     | ..... |
| 系统软键盘                    | ..... |
| 声音开关                     | ..... |
| 截图                       | ..... |
| 录像                       | ..... |
| 切换屏幕                     | ..... |
| 切换用户                     | ..... |
| 锁屏                       | ..... |

|                      |  |
|----------------------|--|
| 关机 and 重启 .....      |  |
| 手势操作指南 .....         |  |
| Android 相关 .....     |  |
| 鼠标控制 .....           |  |
| 键盘控制 .....           |  |
| 5.3 端口转发 .....       |  |
| 5.3.1 连接端口转发 .....   |  |
| 5.3.2 端口转发 API ..... |  |
| 转发通道 .....           |  |
| 5.4 数据传输 .....       |  |
| 5.4.1 连接数据传输 .....   |  |
| 5.4.2 数据传输 API ..... |  |
| 发送数据 .....           |  |

## 一、概述

---

向日葵 iOS 主控 SDK 是上海贝锐信息科技股份有限公司（简称贝锐）开发的一套基于向日葵主控端的 SDK 产品，该 SDK 可以实现在 iOS 应用上快速集成向日葵主控端的主要功能，为应用添加远程桌面。

注意：此文档仅适用于 3.0 版本 SDK，3.0 以下版本请按旧版方法接入。

## 二、SDK 文件说明

---

SDK 文件仅包括 `SunloginControlSDK.framework` 动态库文件。

## 三、集成系统要求

---

SDK 支持最低系统版本为 **iOS11.0**。

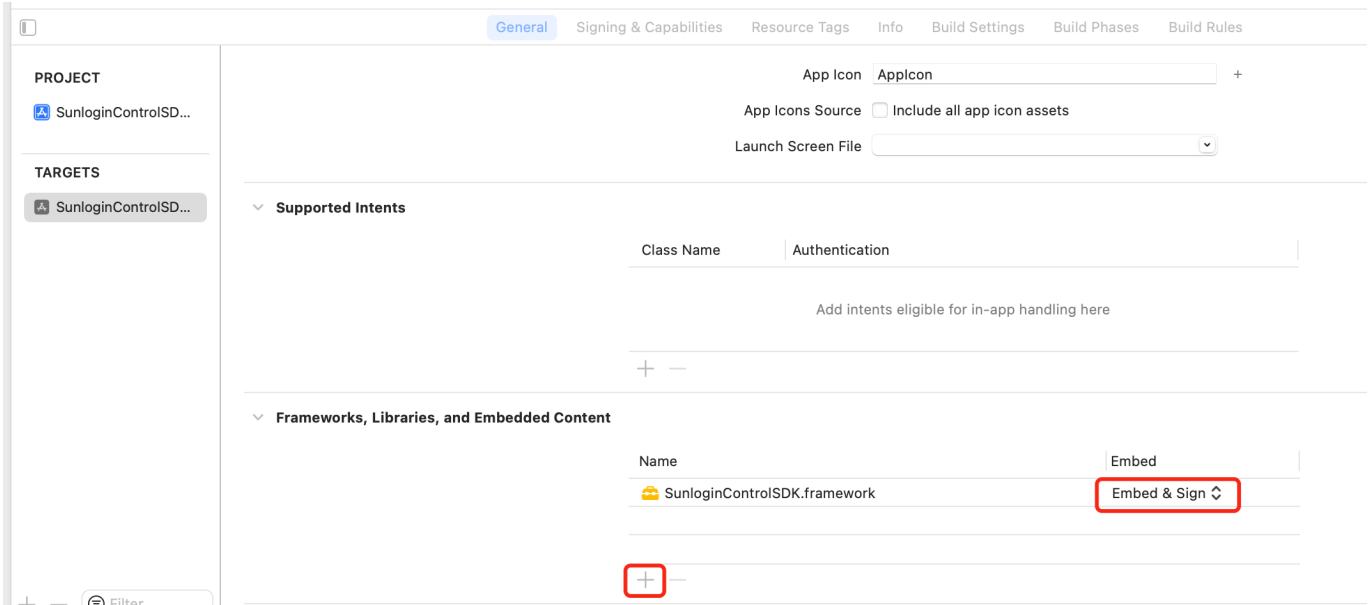
SDK 支持 **arm64** 真机架构。

## 四、集成步骤

---

### 4.1 添加 framework 动态库

`SunloginControlSDK.framework` 为动态库，请选择 **Embed & Sign**。



## 4.2 添加权限

SDK需要申请相机和摄像头权限，需您在工程的`info.plist`文件中添加相应的Key。

```
<plist version="1.0">
<dict>
  <key>NSPhotoLibraryUsageDescription</key>
  <string>是否允许向日葵访问你的照片，用于存储截图和录像？</string>
  <key>NSPhotoLibraryAddUsageDescription</key>
  <string>是否允许向日葵访问你的照片，用于存储截图和录像？</string>
  <key>NSCameraUsageDescription</key>
  <string>是否允许向日葵使用你的相机，用于扫描二维码？</string>
  <key>UIApplicationSceneManifest</key>
</dict>
</plist>
```

| Key                                                 | Type                 | Value                    |
|-----------------------------------------------------|----------------------|--------------------------|
| Information Property List                           | Dictionary (6 items) |                          |
| Appearance                                          | String               | Light                    |
| View controller-based status bar appearance         | Boolean              | YES                      |
| Privacy - Photo Library Usage Description           | String               | 是否允许向日葵访问你的照片，用于存储截图和录像？ |
| Privacy - Photo Library Additions Usage Description | String               | 是否允许向日葵访问你的照片，用于存储截图和录像？ |
| Privacy - Camera Usage Description                  | String               | 是否允许向日葵使用你的相机，用于扫描二维码？   |
| Application Scene Manifest                          | Dictionary (2 items) |                          |

注意：相应的文字描述请您根据自己的app自行修改。

## 五、使用说明

请在使用SDK前引用 `#import <SunloginControlSDK/SunloginControlSDK.h>`。

SDK分为三项主要功能：**远程桌面**、**转发通道**和**数据传输**。

## 5.2 远程桌面

使用从客户端登录后获取的**address**、**session**和**soundSession**连接远程桌面，通过 `SCCDesktopConfig` 配置桌面UI，连接成功后获取 `SCCDesktopController` 桌面控制器，并弹出控制器中的桌面ViewController。

### 5.2.1 配置桌面UI

创建 `SCCDesktopConfig` 对象用来配置桌面UI。 `SCCDesktopConfig` 对象作为每次远控的连接独立参数，并不会被保存记录，每次远控桌面以当此传入的 `SCCDesktopConfig` 对象为准。

```
/// 是否显示状态栏 (default:NO)
@property (assign, nonatomic) BOOL isShowStatusBar;

/// 是否显示默认UI，如关闭需自行调用SCCDesktopController方法，自行绘制菜单等UI
/// (default:YES)
/// 如设置为NO，menuArr、isShowOperationModeBar、isShowDesktopModeBar、
isShowInitiativeAlert、isShowPassivityAlert 均无效
@property (assign, nonatomic) BOOL isEnableUI;

/// 菜单项（期望的菜单项，是否开启控制、被控系统等因素影响最终显示）
@property (copy, nonatomic) NSArray *menuArr;

/// 是否显示操作模式切换栏 (default:YES，包括操作模式切换、虚拟鼠标开关、键盘输入)
@property (assign, nonatomic) BOOL isShowOperationModeBar;

/// 是否显示桌面模式切换栏 (default:YES)
@property (assign, nonatomic) BOOL isShowDesktopModeBar;

/// 是否显示主动断开弹窗 (default:YES，点击退出时的二次确认弹窗)
@property (assign, nonatomic) BOOL isShowActDisconnectAlert;

/// 是否显示被动断开弹窗 (default:YES，被控端关闭或连接中断的提示弹窗，此项关闭时，远控断开
/// 后将直接关闭远程桌面)
@property (assign, nonatomic) BOOL isShowPasDisconnectAlert;
```

注意：如不想使用SDK自带默认UI，请设置isEnableUI为**NO**，远程桌面将隐藏全部功能菜单和弹窗交互，您可通过 `SCCDesktopController` 中的API手动实现功能。

`menuArr` 属性设置的菜单项显示，会根据远控类型、被控系统进行自动判断，请将全部需要的菜单项设置放入数组，数组的顺序将会决定菜单项的显示顺序。

```
SCCDesktopConfig *config = [[SCCDesktopConfig alloc] init];
// 显示状态栏
config.isShowStatusBar = NO;
// 使用自带UI
config.isEnableUI = YES;
// 菜单项: 退出、声音开关、关机
config.menuArr = @[__RPMMenu_Quit, __RPMMenu_Sound, __RPMMenu_Shutdown];
// 显示切换操作栏
config.isShowOperationModeBar = YES;
// 显示切换桌面模式栏
config.isShowDesktopModeBar = YES;
// 显示主动断开弹窗
config.isShowActDisconnectAlert = YES;
// 显示被动断开弹窗
config.isShowPasDisconnectAlert = YES;
```

### 5.2.2 连接远程桌面

使用被控客户端获取的**address**桌面、声音**session**连接远程桌面。**session**和 **soundSession**连接过一次后即失效，下次远控请在客户端重新生成。

```

/// 连接远程桌面方法，成功后回调 SCCDesktopController 远程桌面控制器
/// - **Parameters**:
/// - address: 客户端生成的 address
/// - session: 客户端生成的 session
/// - soundSession: 客户端生成的声音 session （可选，如无需连接声音可不传）
/// - system: 客户端系统
/// - desktopConfig: 桌面样式配置
/// - delegate: 状态代理
/// - complete: 连接成功后回调 SCCDesktopController 远程桌面控制器
[[SCCSDK sdk] connectRemoteDestopWithAddress:address
                                session:session
                                soundSession:soundSession
                                system:self.remoteSystem
                                desktopConfig:self.desktopConfig
                                delegate:self
                                complete:^(SCCDesktopController * _Nonnull
desktopController) {
    // 连接远程桌面成功
    self.desktopController = desktopController; // 持有桌面控制器
    // 设置代理
    [self.desktopController setDesktopDelegate:self];
    // 取出桌面 ViewController
    UIViewController *vc = [desktopController getDesktopViewController];
    if (vc) {
        // 弹出远程桌面页
        vc.modalPresentationStyle = UIModalPresentationFullScreen; // 覆盖全屏
        [self presentViewController:vc animated:YES completion:^(
            self.sessionTextField.text = nil;
            self.soundSessionTextField.text = nil;
        )];
    }
}];
}];

```

注意：SDK无法获取被控系统类型，桌面区分电脑或手机依赖 `system` 参数，请保证设置正确。

连接状态将通过 `<SCCSDKConnectStateDelegate>` 返回

```

/// 连接状态回调
@protocol SCCSDKConnectStateDelegate <NSObject>
@optional
/// 正在连接
- (void)SCCSDKConnecting;
/// 即将连接成功
- (void)SCCSDKConnectWillSucceed;
/// 连接成功
- (void)SCCSDKConnectSucceed;
/// 连接失败
- (void)SCCSDKConnectFailedWithErrorCode:(NSInteger)errorCode errorMessage:
(NSString *)errorMessage;
@end

```

设置桌面状态代理

```
[self.desktopController setDesktopDelegate:self];
```

获取远程桌面viewController

```
UIViewController *vc = [desktopController getDesktopViewController];
```

注意：登录成功后会返回 `SCCDesktopController` 远程桌面控制器，请在远控期间持有控制器。

远控结束后请释放 `SCCDesktopController` 控制器，不释放控制器，远程桌面的 **ViewController** 将无法释放。

### 5.2.3 远程桌面控制API

在关闭默认UI的情况下，通过调用桌面控制器API，可实现相应功能。

远程桌面状态

将通过 `<SCCDesktopControllerDelegate>` 返回。

```

/// 远程桌面页状态回调
@protocol SCCDesktopControllerDelegate <NSObject>
/// 远程桌面断开
/// - Parameter isActive: 是否主动断开
- (void)SCCDesktopDisconnect:(BOOL)isActive;
/// 远程桌面已显示
- (void)SCCDesktopDidAppear;
/// 远程桌面已消失
- (void)SCCDesktopDidDisappear;
@end

```

## 基础控制

```

/// 关闭远程桌面
- (void)closeDesktop;
/// 暂停接收数据，建议在app进入后台后调用
- (void)pauseReceiveData;
/// 恢复数据传输，建议在app恢复前台后调用
- (void)resumeReceiveData;

```

## 桌面模式

远控Windows时可切换桌面模式。

```

/// 桌面模式类型（传输模式）
typedef NS_ENUM(NSInteger, SCCDesktopTransferMode) {
    /// 均衡模式（画质与带宽占用均衡）
    SCCDesktopTransferModeNormal = 0,
    /// 娱乐模式（更高的帧率、画面质量和带宽占用）
    SCCDesktopTransferModeEntertain,
    /// 极速模式（黑白画面，最低带宽占用）
    SCCDesktopTransferModeTopSpeed
};

```

```

/// 当前桌面模式
- (SCCDesktopTransferMode)currentTransferMode;
/// 切换桌面模式
/// - Parameter transferMode: 桌面模式
- (void)switchTransferMode:(SCCDesktopTransferMode)transferMode;

```

## 操作模式

切换触摸模式或指针模式。

```
/// 桌面模式类型（操作模式）
typedef NSInteger (NS_ENUM(SCCDesktopOperationMode)) {
    /// 触摸模式
    SCCDesktopOperationModeTouch = 0,
    /// 指针模式
    SCCDesktopOperationModeCursor
};
```

```
/// 当前操作模式
- (SCCDesktopOperationMode)currentOperationMode;
/// 切换操作模式
/// - Parameter operationMode: 操作模式
- (void)switchOperationMode:(SCCDesktopOperationMode)operationMode;
```

## 虚拟鼠标

远控PC触摸模式下可切换是否显示虚拟鼠标。

```
/// 当前是否显示虚拟鼠标（仅PC触摸模式提供）
- (BOOL)isShowVirtualMouse;
/// 切换是否显示虚拟鼠标（仅PC触摸模式有效）
/// - Parameter isShowVirtualMouse: 虚拟鼠标显示
- (void)switchIsShowVirtualMouse:(BOOL)isShowVirtualMouse;
```

## 系统软键盘

```
/// 切换是否显示键盘
/// - Parameter isShow: 键盘显示
- (void)switchIsShowKeyboard:(BOOL)isShow;
```

## 声音开关

在传声音**session**时可连接被控声音，首次连接时间可能会较长。

```

/// 当前声音开关状态
- (BOOL)currentIsSoundOn;
/// 切换声音开关，需配置sound session，首次开启声音时间可能会较长，建议增加UI提示
/// - Parameters:
///     - isOn: 打开或关闭声音
///     - complete: 完成操作后回调
- (void)switchSound:(BOOL)isOn complete:(void(^)(BOOL isSucceed, BOOL isSoundOn))complete;

```

## 截图

```

/// 截图
/// - Parameter complete: 完成操作后回调
- (void)getScreenshotComplete:(void(^)(UIImage *screenshot))complete;

```

## 录像

```

/// 开始录制，调用 recordScreenEnd 结束，占用内存过大时，可能会自动中断
/// - Parameters:
///     - filePath: 录制文件路径
///     - complete: 完成操作后回调
- (void)recordScreenToFilePath:(NSString *)filePath complete:(void(^)(BOOL isSucceed, NSString *_Nullable filePath))complete;
/// 结束录制
- (void)recordScreenEnd;

```

## 切换屏幕

被控连接多个屏幕时，可调用切换。

```

/// 切换屏幕
/// - Parameter complete: 完成操作后回调
- (void)switchScreenComplete:(void(^)(void))complete;

```

## 切换用户

被控登录多个用户时，可调用切换。

```

/// 切换用户
- (void)switchConsoleUser;

```

## 锁屏

部分被控可能无法通过api解锁屏幕。

```
/// 切换电脑锁屏状态  
/// - Parameter isLock: 锁屏或解锁  
- (void)switchDesktopLock:(BOOL)isLock;
```

## 关机和重启

```
/// 关机  
- (void)shutdown;  
/// 重启  
- (void)reboot;
```

## 手势操作指南

```
/// 显示操作指南  
- (void)showGuideView;
```

## Android相关

远控安卓手机时生效

```
/// Android 下拉
- (void)androidDropDown;
/// Android 上拉
- (void)androidPullUp;

/// Android 点击返回键
- (void)androidClickBack;
/// Android 点击home键
- (void)androidClickHome;
/// Android 点击菜单键
- (void)androidClickMenu;

/// Android 音量+
- (void)androidVoiceUp;
/// Android 音量-
- (void)androidVoiceDown;

/// Android 屏幕锁屏
- (void)androidScreenLock;

/// Android 切换横屏
- (void)androidLandscape;
/// Android 切换竖屏
- (void)androidVertical;
```

## 鼠标控制

point鼠标坐标相对于桌面ViewController.view。

```

/// 鼠标左键按下, 坐标位于远程桌面ViewController.view
/// - Parameter point: 鼠标坐标
- (void)mouseLeftDownAtPoint:(CGPoint)point;
/// 鼠标左键抬起, 坐标位于远程桌面ViewController.view
/// - Parameter point: 鼠标坐标
- (void)mouseLeftUpAtPoint:(CGPoint)point;

/// 鼠标右键按下, 坐标位于远程桌面ViewController.view
/// - Parameter point: 鼠标坐标
- (void)mouseRightDownAtPoint:(CGPoint)point;
/// 鼠标右键抬起, 坐标位于远程桌面ViewController.view
/// - Parameter point: 鼠标坐标
- (void)mouseRightUpAtPoint:(CGPoint)point;

/// 鼠标中键按下, 坐标位于远程桌面ViewController.view
/// - Parameter point: 鼠标坐标
- (void)mouseMidDownAtPoint:(CGPoint)point;
/// 鼠标中键抬起, 坐标位于远程桌面ViewController.view
/// - Parameter point: 鼠标坐标
- (void)mouseMidUpAtPoint:(CGPoint)point;

/// 鼠标移动到位置, 坐标位于远程桌面ViewController.view
/// - Parameter point: 鼠标坐标
- (void)mouseMoveToPoint:(CGPoint)point;

/// 鼠标滚轮准备, 坐标位于远程桌面ViewController.view
/// - Parameter point: 鼠标坐标
- (void)mouseWheelStartAtPoint:(CGPoint)point;

/// 鼠标滚轮滚动, 坐标位于远程桌面ViewController.view
/// - Parameters:
///   - point: 鼠标坐标
///   - angle: 单次滚动角度 例如向下90度则传入90, 向上90度则传入-90
///   - isHorizon: 横向或纵向滚动
- (void)mouseWheelAtPoint:(CGPoint)point angle:(CGFloat)angle isHorizon:
(BOOL)isHorizon;

```

## 键盘控制

keyName键值详见 `SCCDesktopConfig`

```

/// 键盘按下，按键值详见 SCCDesktopConfig
/// - Parameter keyName: 按键值
- (void)keyboardDownWithKey:(NSString *)keyName;
/// 键盘抬起，按键值详见 SCCDesktopConfig
/// - Parameter keyName: 按键值
- (void)keyboardUpWithKey:(NSString *)keyName;

/// 直接发送字符串
/// - Parameter string: 发送字符串
- (void)sendMessageString:(NSString *)string;

```

## 5.3 端口转发

连接转发端口后，可创建通道，通过返回的地址可实现端口转发。

### 5.3.1 连接端口转发

```

// 连接端口转发
[[SCCSDK sdk] connectPortForwardWithAddress:address session:session ip:ipStr
delegate:self];

// 断开端口转发
[[SCCSDK sdk] disconnectPortForward];

```

连接状态将通过 `<SCCSDKConnectStateDelegate>` 返回

```

/// 连接状态回调
@protocol SCCSDKConnectStateDelegate <NSObject>
@optional
/// 正在连接
- (void)SCCSDKConnecting;
/// 即将连接成功
- (void)SCCSDKConnectWillSucceed;
/// 连接成功
- (void)SCCSDKConnectSucceed;
/// 连接失败
- (void)SCCSDKConnectFailedWithErrorCode:(NSInteger)errorCode errorMessage:
(NSString *)errorMessage;
@end

```

### 5.3.2 端口转发API

## 转发通道

```
/// 创建转发通道
/// - Parameters:
///   - port: 端口
///   - complete: 完成回调
- (void)createChannelWithPort:(NSUInteger)port complete:(void(^)(BOOL
isSucceed, NSString *ipAndPort))complete;

/// 销毁一个已创建的转发通道
- (void)deleteChannelWithPort:(NSUInteger)port;
```

## 5.4 数据传输

连接数据传输后，可发送数据到被控和从被控接收数据。

### 5.4.1 连接数据传输

```
// 连接数据传输
[[SCCSDK sdk] connectDataTransferWithAddress:address session:session ip:ipStr
delegate:self recvDataComplete:^(NSString * _Nonnull recvData) {
    self.stateLabel.text = recvData;
    NSLog(@"接收数据: %@", recvData);
}];

// 断开数据传输
[[SCCSDK sdk] disconnectDataTransfer];
```

连接状态将通过 <SCCSDKConnectStateDelegate> 返回

```
/// 连接状态回调
@protocol SCCSDKConnectStateDelegate <NSObject>
@optional
/// 正在连接
- (void)SCCSDKConnecting;
/// 即将连接成功
- (void)SCCSDKConnectWillSucceed;
/// 连接成功
- (void)SCCSDKConnectSucceed;
/// 连接失败
- (void)SCCSDKConnectFailedWithErrorCode:(NSInteger)errorCode errorMessage:
(NSString *)errorMessage;
@end
```

## 5.4.2 数据传输API

### 发送数据

```
/// 发送数据  
/// - Parameter data: 发送的信息  
- (void)sendData:(NSString *)data;
```